Capital University

Faculty of Computing & Artificial Intelligence

Wiggle Sort

Algorithm Project

Prepared by: Mega Team

Table of Contents

1. Introduction

In this project, we implement and analyze a specific variation of sorting known as Wiggle Sort.

The Wiggle Sort algorithm rearranges elements in an array such that the elements follow an alternating pattern:

Elements at even indices are less than or equal to their next element.

Elements at odd indices are greater than or equal to their next element.

This project presents two different implementations of the Wiggle Sort algorithm:

A recursive approach, which uses function calls to achieve the desired ordering.

A non-recursive (iterative) approach, which uses loops for direct execution.

Both implementations are analyzed in terms of time complexity, space complexity, efficiency, and overall performance. The main objective of this project is to compare recursion and iteration in solving the same problem and understand the trade-offs between them in terms of memory usage and execution efficiency.

2. First algorithm (Recursive)

2.1 pseudocode:

```
function swap(i, j)

  temp ← value at i
  value at i ← value at j
  value at j ← temp


function wigglesort(arr, n, i)

  if i ≥ n - 1 then
    return


  if i mod 2 = 0 and arr[i] > arr[i + 1] then
    swap(arr[i], arr[i + 1])
```

```
  if i mod 2 ≠ 0 and arr[i] < arr[i + 1] then
    swap(arr[i], arr[i + 1])


  wigglesort(arr, n, i + 1)
```

2.2 implementation:

```
void swap(int *a, int *b){
  int temp = *a;
  *a = *b;
  *b = temp;
}

void wiggleSort(int arr[], int n, int i){
  if(i >= n-1)
    return;
  if(i % 2 == 0 && arr[i] > arr[i+1])
    swap(&arr[i], &arr[i+1]);
  else if(i % 2 != 0 && arr[i] < arr[i+1])
    swap(&arr[i], &arr[i+1]);
  wiggleSort(arr, n, i+1);
}
```

2.3 Analysis:

Start from index i
If i ≥ n - 1, stop recursion
For each index:
If i is even and arr[i] > arr[i+1] → swap
If i is odd and arr[i] < arr[i+1] → swap
Recursively call the function for i + 1

2.4 Complexity:

2.4.1 Time

Recurrence relation:

$$T(n) = T(n-1) + O(1)$$

Using Iteration Method:

$$T(n) = T(n-1) + O(1)$$
$$T(n) = (T(n-2) + O(1)) + O(1)$$
$$T(n) = T(n-2) + 2O(1)$$
$$T(n) = T(n-3) + 3O(1)$$

After k iterations:

$$T(n) = T(n-k) + kO(1)$$

At the base case:

$$n - k = 1$$
$$k = n - 1$$

Substitute:

$$T(n) = T(1) + (n-1)O(1)$$

Since:

$$T(1) = O(1)$$

Therefore:

$$T(n) = O(n)$$

2.4.2 Space

Space complexity = O(n)

3. Second algorithm (non-recursive):

3.1 pseudocode:

```
function swap(i, j)
  temp ← value at i
  value at i ← value at j
  value at j ← temp
function wigglesort(arr, n)
  for i ← 0 to n - 2 do

    if i mod 2 = 0 and arr[i] > arr[i + 1] then
      swap(arr[i], arr[i + 1])

    if i mod 2 ≠ 0 and arr[i] < arr[i + 1] then
      swap(arr[i], arr[i + 1])
```

3.2 Implementation:

```
void wiggleSort(int arr[], int n){
  for(int i = 0; i < n-1; i++){
    if(i % 2 == 0 && arr[i] > arr[i+1])
      swap(&arr[i], &arr[i+1]);
    else if(i % 2 != 0 && arr[i] < arr[i+1])
      swap(&arr[i], &arr[i+1]);
  }
}
```

3.3 Analysis:

At even indices (i % 2 == 0):
→ arr[i] ≤ arr[i + 1]
At odd indices (i % 2 != 0):
→ arr[i] ≥ arr[i + 1]

The algorithm works by:
Traversing the array once from i to n - 2
Comparing adjacent elements
Swapping them if they do not satisfy the wiggle condition

3.4 Complexity:

3.4.1 Time:

the loop runs from 0 to n – 2
so, the number of iterations is proportional to n
Time complexity = O(n)

- $T(n) = \sum_{\{i=0\}}^{\{n-2\}} O(1) = O(n)$
- Best case: O(n)
- Average case: O(n)
- Worst case: O(n)

3.4.2 Space:

Space complexity = O(1)

4. Comparison:

Feature	Recursive	Non-Recursive
Approach	Uses function calls	Uses loop
Control Flow	Call stack-based execution	Linear Execution
Time Complexity	O(n)	O(n)
Space Complexity	O(n)	O(1)
Efficiency	Less efficient	More efficient
Risk	Possible stack overflow for large n	No Stack overflow
Readability	More elegant but slightly harder to trace	Simple & direct